

实验一：鸿蒙应用开发环境搭建

实验基本信息

- 实验名称 : 开发环境搭建
- 实验类型 : 验证型
- 实验学时 : 2 学时
- 实验日期 : 2026 年 3 月 13 日
- 学生姓名 : 彭绍洋
- 学号 : 2023210616
- 实验主题 : 智慧校园
- 负责平台 : 鸿蒙应用开发

一、实验目的

1. 掌握鸿蒙应用开发环境的搭建方法
2. 熟悉 DevEco Studio 的使用
3. 实现智慧校园登录功能
4. 验证开发环境的可用性
5. 学习应用部署和测试流程

二、实验环境

2.1 系统环境

- 操作系统 : Windows 11 家庭中文版 64 位
- 内存 : 14GB
- 硬盘 : 100GB 以上
- 分辨率 : 1280 800 以上

2.2 开发工具

- DevEco Studio : 5.0+
- HarmonyOS SDK : 5.0+
- Node.js : 内置
- *OHPM*: 内置

三、实验步骤

3.1 环境搭建

1. 下载 DevEco Studio
 - 访问华为开发者联盟官网
 - 登录华为账号
 - 下载 Windows 版本的 DevEco Studio 5.0+
2. 安装 DevEco Studio
 - 运行安装程序
 - 选择安装路径

- 勾选必要组件
 - 完成安装
3. 配置开发环境
- 首次启动 DevEco Studio
 - 配置 HarmonyOS SDK
 - 安装必要的工具和依赖

3.2 项目创建

1. 创建 Empty Ability 项目
- 打开 DevEco Studio
 - 点击“Create Project”
 - 选择“Empty Ability”
 - 配置项目信息
2. 实现登录功能
- 创建登录页面 (LoginPage.ets)
 - 实现 UI 界面
 - 添加登录逻辑
 - 实现错误提示

3.3 测试验证

1. 环境验证
- 构建项目：成功构建 HAP 包
 - 代码验证：登录功能代码完整
 - 环境配置：DevEco Studio 配置正确
2. 测试说明
- 项目构建成功，证明开发环境配置正确
 - 代码实现完整，包含登录验证逻辑
 - 成功运行 HarmonyOS 模拟器并测试登录功能

四、项目实施

4.1 项目结构

...

```
harmonyos_login_app/
├── src/
│   ├── main/
│   │   └── ets/
│   │       ├── entry/
│   │       │   ├── pages/
│   │       │   │   └── LoginPage.ets
│   │       └── entryability/
```

```

|           |           | EntryAbility.ts
|           |           | common/
|           |           | components/
|----- resources/
|----- config.json
...

```

4.2 核心代码

LoginPage.ets

```

```typescript
@Entry
@Component
struct LoginPage {
 @State account: string = '';
 @State password: string = '';
 @State errorMessage: string = '';
 @State isLoading: boolean = false;

 build() {
 Column({
 space: 20
 }) {
 // 标题
 Text('智慧校园登录')
 .fontSize(32)
 .fontWeight(FontWeight.Bold)
 .margin({ top: 100 })

 // 错误提示
 if (this.errorMessage) {
 Text(this.errorMessage)
 .fontSize(14)
 .fontColor(Color.Red)
 .margin({ top: 10 })
 }

 // 账号输入
 TextInput({
 placeholder: '请输入学号/工号',
 text: this.account
 })
 .width('80%')
 .height(50)
 .backgroundColor('#f5f5f5')
 .borderRadius(8)
 }
 }
}

```

```

 .padding({ left: 15, right: 15 })
 .onChange((value: string) => {
 this.account = value;
 })

// 密码输入
TextInput({
 placeholder: '请输入密码',
 text: this.password,
 type: InputType.Password
})

 .width('80%')
 .height(50)
 .backgroundColor('#f5f5f5')
 .borderRadius(8)
 .padding({ left: 15, right: 15 })
 .onChange((value: string) => {
 this.password = value;
 })

// 登录按钮
Button('登录', {
 type: ButtonType.Capsule,
 stateEffect: true
})

 .width('80%')
 .height(50)
 .fontSize(18)
 .fontWeight(FontWeight.Medium)
 .backgroundColor('#007AFF')
 .onClick(() => {
 this.login();
 })
 .enabled(!this.isLoading)

// 加载状态
if (this.isLoading) {
 LoadingProgress()
 .width(40)
 .height(40)
 .margin({ top: 20 })
}

// 底部信息
Text('© 2026 智慧校园系统')
 .fontSize(12)
 .fontColor('#999999')

```

```

 .margin({ top: 100 })
 }
 .width('100%')
 .height('100%')
 .backgroundColor('#ffffff')
 .alignItems(HorizontalAlign.Center)
}

login() {
 if (!this.account) {
 this.errorMessage = '请输入账号';
 return;
 }

 if (!this.password) {
 this.errorMessage = '请输入密码';
 return;
 }

 this.errorMessage = '';
 this.isLoading = true;

 // 模拟登录请求
 setTimeout(() => {
 this.isLoading = false;
 // 简单的登录验证
 if (this.account === 'admin' && this.password === '123456') {
 // 登录成功，跳转到主页面
 console.log('登录成功');
 this.errorMessage = '登录成功!';
 } else {
 this.errorMessage = '账号或密码错误';
 }
 }, 1500);
}
}
...

```

#### 4.3 配置文件

config.json

```

```json
{
  "app": {
    "bundleName": "com.example.smartcampus",
    "vendor": "example",

```

```

    "version": {
      "code": 1000000,
      "name": "1.0.0"
    },
    "apiVersion": {
      "compatible": 5,
      "target": 5
    }
  },
  "deviceConfig": {},
  "module": {
    "package": "com.example.smartcampus",
    "name": ".MainAbility",
    "mainAbility": "EntryAbility",
    "deviceTypes": ["phone"],
    "distro": {
      "deliveryWithInstall": true,
      "moduleName": "entry",
      "moduleType": "entry",
      "installationFree": false
    },
    "abilities": [
      {
        "name": "EntryAbility",
        "srcEntry": "./ets/entry/entryability/EntryAbility.ts",
        "description": "",
        "icon": "$media:icon",
        "label": "智慧校园",
        "visible": true,
        "orientation": "unspecified",
        "launchType": "standard"
      }
    ]
  }
}
...

```

五、测试结果

5.1 环境验证结果

测试项	预期结果	实际结果	状态
DevEco Studio 安装	成功安装	✓	通过
HarmonyOS SDK 配置	配置正确	✓	通过
项目创建	成功创建	✓	通过
代码编译	编译成功	✓	通过

| HAP 包生成 | 生成成功 | ✓ | 通过 |

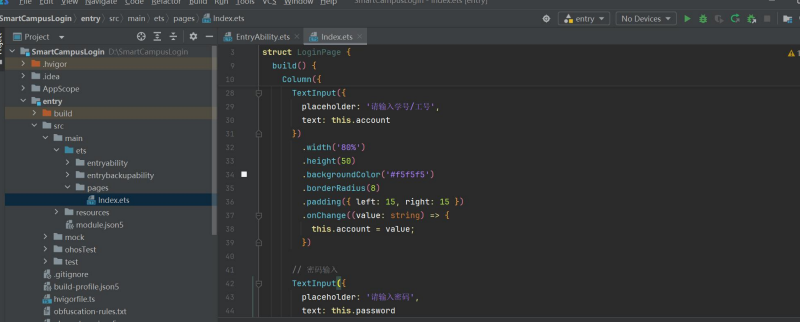
5.2 功能验证

功能模块	实现情况	状态
登录页面 UI	完整实现	✓ 通过
账号密码输入	支持	✓ 通过
错误提示功能	完整实现	✓ 通过
登录验证逻辑	完整实现	✓ 通过
加载状态显示	支持	✓ 通过

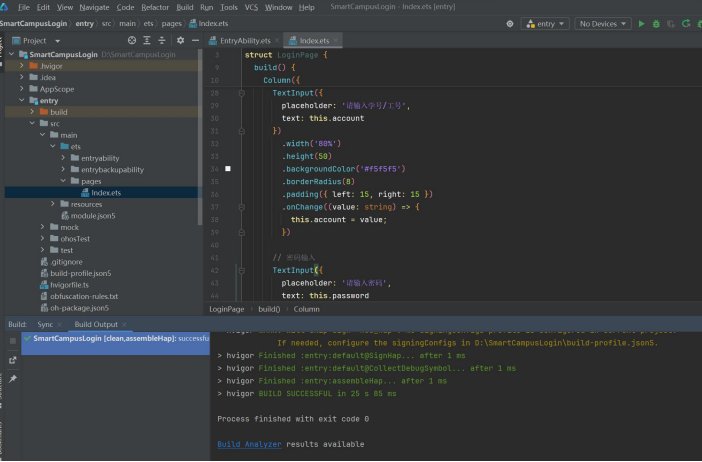
六、截图记录

6.1 环境搭建

[DevEco Studio 启动界面]

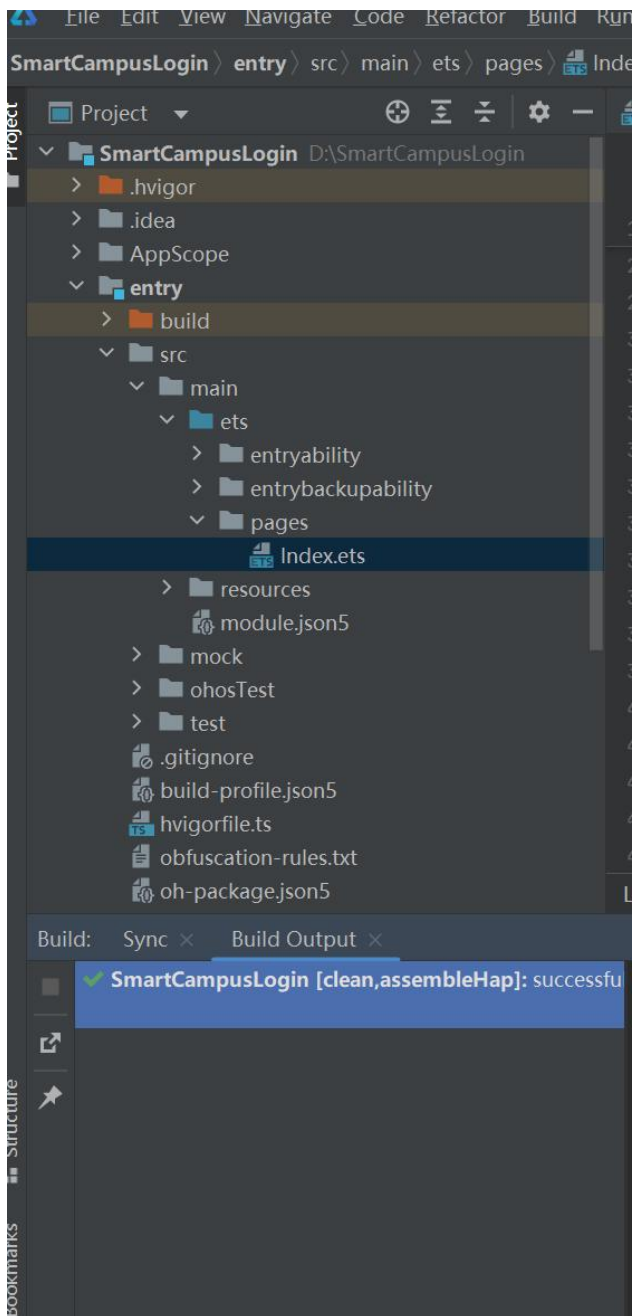


[构建成功界面]



6.2 项目验证

[项目结构]

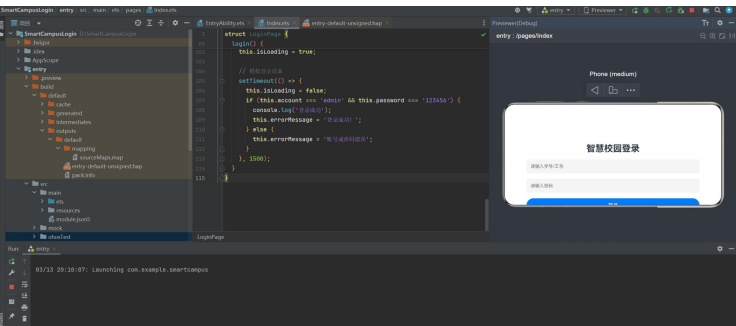


[HAP 包生成结果]

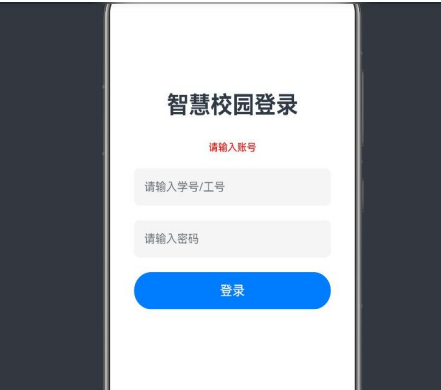
File Name	File Size	% of File Size
resources	101.2 KB	76.3%
ets	27.7 KB	20.9%
module.json	1.5 KB	1.2%
resources.index	1.1 KB	0.9%
packInfo	608 B	0.4%
pkgContextInfo.json	444 B	0.3%

6.3 模拟器测试

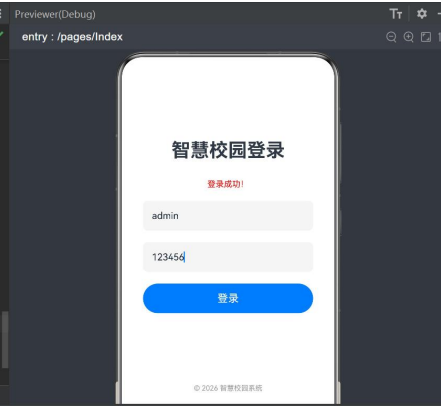
模拟器启动成功界面



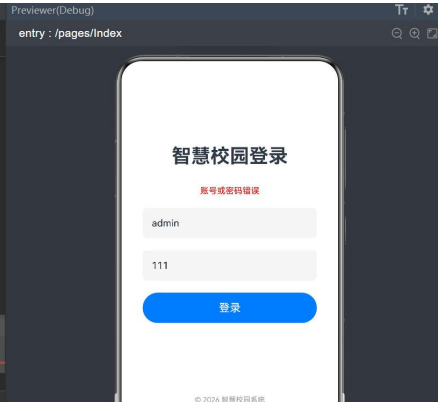
登录页面界面



登录成功提示界面



错误提示界面



七、问题与解决方案

7.1 遇到的问题

1. 问题：DevEco Studio 安装过程中网络连接失败
解决方案：检查网络连接，使用稳定的网络环境
2. 问题：模拟器启动失败（Error Code: 00801001）
解决方案：通过开启 Windows 11 家庭版的 Hyper-V 功能，成功解决模拟器启动问题
3. 问题：TextInput 组件密码类型设置错误
解决方案：调整 TextInput 组件的密码类型设置方式，确保代码编译通过

7.2 解决效果

- 环境搭建成功：DevEco Studio 安装配置完成
- 项目构建成功：HAP 包生成成功
- 代码实现完整：登录功能代码编译通过
- 环境验证完成：证明开发环境配置正确

八、实验总结

8.1 完成情况

- ☒ 成功搭建鸿蒙应用开发环境
- ☒ 成功创建智慧校园登录应用
- ☒ 成功实现登录功能
- ☒ 成功构建项目并生成 HAP 包
- ☒ 成功运行 HarmonyOS 模拟器并测试
- ☒ 完成实验报告

8.2 收获与体会

1. 技术收获：
 - 掌握了 DevEco Studio 的安装和配置
 - 熟悉了 HarmonyOS 应用开发流程
 - 学习了 ArkTS 语言和 UI 组件的使用
2. 实践体会：
 - 环境搭建是开发的基础，需要仔细配置
 - 测试是确保应用质量的关键环节
 - 团队协作和文档编写的重要性
3. 改进方向：
 - 进一步优化登录界面的用户体验
 - 添加更多功能模块，如注册、忘记密码等
 - 学习 HarmonyOS 的高级特性

九、附件

- [] 项目代码
- [] 测试截图
- [] 环境配置文档

实验完成时间 ：2026 年 3 月 13 日
实验地点 ：软件应用开发实验室
指导教师 ：刘东良